=

Computer Science

Internal Assessment

Family Medication Management System

Word count: 1954

Criterion A - Problem Specification

Problem Scenario

One day in May, “Have you taken your meds today?” Mom asked Dad. “Oh, have I? I might have, but I forgot…” Then, a couple of days later, “damn, this med is out of stock!” Dad shouted, “How come we did not know?” Mom said, “Today I just found my vitamin D capsules had expired in March, and I have been taking them.”

The above incidents seem to occur increasingly frequent nowadays. So when I expressed the desire to do something about this in the context of my IB CS IA, my newest clients (Mom and Dad) gave a strong vote “YES”. (Appendix-A)

Computational Context

My proposed solution will feature a GUI-based standalone application that will work on a home Local-Area-Network (LAN) and personal laptop/PC. Java is chosen for its strong OOP support, powerful swing framework for GUI, networking libraries and JDBC support. SQLite will be the DB framework for its elegant and native support for Java and fits our light-load scenario. The system will support the following:

- A GUI that connects to the family backend service on the home LAN, supporting Management of the family medication repository: track and remind family members of medication schedules, checking expiry dates, give warnings on low-stock medications.
- The same GUI can connect to an individual family member’s backend service on his own PC, which can travel with the member and manage the meds in his/her individual repository on the road.
- Syncing between the individual repositories and the family repository when they are on the home LAN, supporting withdrawing and merging scenarios.
- A user has the option to track his schedule/dosage online.

[Word count: 274]

Success criteria (Client session in APPENDIX-C)

ID	Measurable success criterion
1	From the bilingual GUI, an authenticated family user can select the family repository, connect, choose a username from a dropdown, log in, and view or manage the shared medication grid at home. The very first user is created during the family repo's first-run setup.
2	When a brand-new family repository has no users, the GUI's "Create first admin" dialog makes one account – and that bootstrap user is forced to be an admin and becomes the repository owner. (This is the one time a user can be created without already being logged in.) Every user after that is created by a logged-in family admin via Repository → Manage Users → Add (add / edit / delete, and set who's an admin).
3	The same GUI can connect to a dedicated or shared personal repository: family users can log in offline, manage travelling medication, and later synchronize it with the family repository after approval.
4	A medication batch stores name, dosage, daily doses, quantity, expiry date, low-stock threshold, Global Trade Item Number(GTIN), drug code and optional 2D-code text; English and Chinese text round-trips unchanged.
5	A logged-in user can create, edit and delete only that user's acute, temporary or chronic schedules, including condition, doses per day, quantity per dose and valid start/end dates.
6	For every active schedule the dashboard displays TAKEN, DUE, MISSED or INACTIVE and issues a missed-dose reminder after an evenly spaced dose slot between 08:00 and 22:00 passes unmet.
7	Recording a scheduled dose atomically subtracts the requested quantity from the earliest-expiry matching batch with enough stock and creates a server-timestamped DoseLog linked to the schedule.
8	The system automatically calculates remaining quantity and a run-out date of today plus quantity divided by daily doses; as-needed medication shows no forecast.
9	The dashboard and reports sort batches by expiry and display expired and low-stock warnings when expiry is before today or quantity is at or below its threshold.
10	Family administrators can manage users; repository owners or administrators can change repository settings; passwords are salted and hashed; medication add/edit/delete actions record username, action, detail and timestamp.
11	A family user can withdraw a chosen batch quantity to a personal repository; matching stock and the personal owner's relevant schedules move, the personal status becomes TRAVELLING, and a failed deposit restores family stock.
12	While travelling, users can record doses and add newly obtained medication in the personal repository, including when several copied family users share that repository.

ID	Measurable success criterion
13	The GUI blocks family-repository dose recording when the local personal repository is TRAVELLING, and two present users can approve a stock transfer between personal repositories.
14	On return, synchronization previews personal batches, then on approval merges batches by name, dosage and expiry, preserves portable schedules and dose timestamps, clears the personal records, and sets it HOME.
15	When a family repository starts, UDP discovery checks the LAN and refuses startup if another discoverable family repository is already running.
16	A paired member (through GUI to website) can record a scheduled dose in the website; the GUI then applies that tap once to the repository holding the matching schedule, keeps the web-entry's timestamp, subtracts stock, and marks the tap synced, rejected, or still waiting.
17	Overall GUI experience and Web experience seamless.

Criterion B: Planning

Decomposing Problems

I decomposed the overall problem of family medication management into smaller computational problems that could be developed and tested independently.

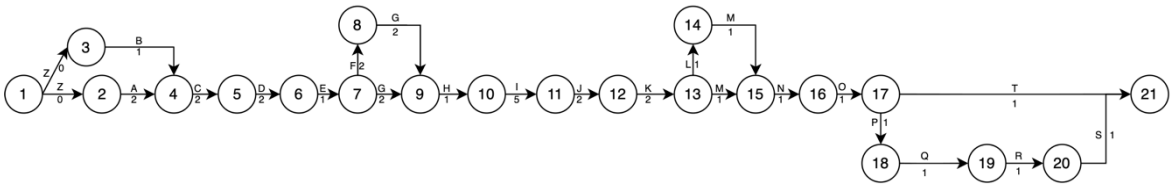
- Medication management: store medications and batches, track quantities and expiry dates, and generate low-stock or expiry warnings.
- Scheduling: create user-specific schedules, calculate dose times, and give alerts for missed dosages.
- Dose recording: identify the schedule, check available stock, select the appropriate batch, deduct the dose, and record a dose record.
- User management: authenticate users, assign roles, and control access to personal and family data.
- Travelling: withdraw/transfer relevant medication and schedules, allow offline changes, and synchronize them after returning.
- Web recording: receive, validate, queue, and synchronize web-based dose records.

These subproblems were then organized by their dependencies and used to plan the development schedule.

Planning the Computational Thinking Process

- PERT Chart:

Activity	Description	Weeks to complete	Deliverables	Milestone
			Start	1
A	Making IA idea mindmaps	2	Problem brainstorming	2
B	Asking for more information + gathering ideas	1	Meeting with client	3
C	Present my possible solutions + specify requirements	2	Meeting with client #2	4
D	First draft of product overview + UI diagram	2	Planning	5
E	presented overall design + feedback	1	Meeting with client #3	6
F	Changing UI diagram based on feedback + flowcharts	2	Planning	7
G	Development timeline	2	Planning	8
H	Creating a wire frame diafram	1	Planning	9
I	GUI and repository service implementation	5	Building Database	10
J	Asking for more requirements	2	Meeting with client #4	11
K	Learning SQLite	2	Building Database	12
L	UDP & TCP	2	Building Repository	13
M	Present demo. + clearify the requirements	1	Meeting with client #5	14
N	Scanning Barcode to record medication information	1	Program development	15
O	Implementating the Schedule for daily medicine intake	1	Program development	16
P	Record for medicine intake + Audit log	1	Program development	17
Q	Alerts system	1	Program development	18
R	Test	1	Test	19
S	Pilot Deployment	1	Pilot Deployment	20
T	seeking client feedback for the Pilot Deployment	1	Meeting with client #6	21

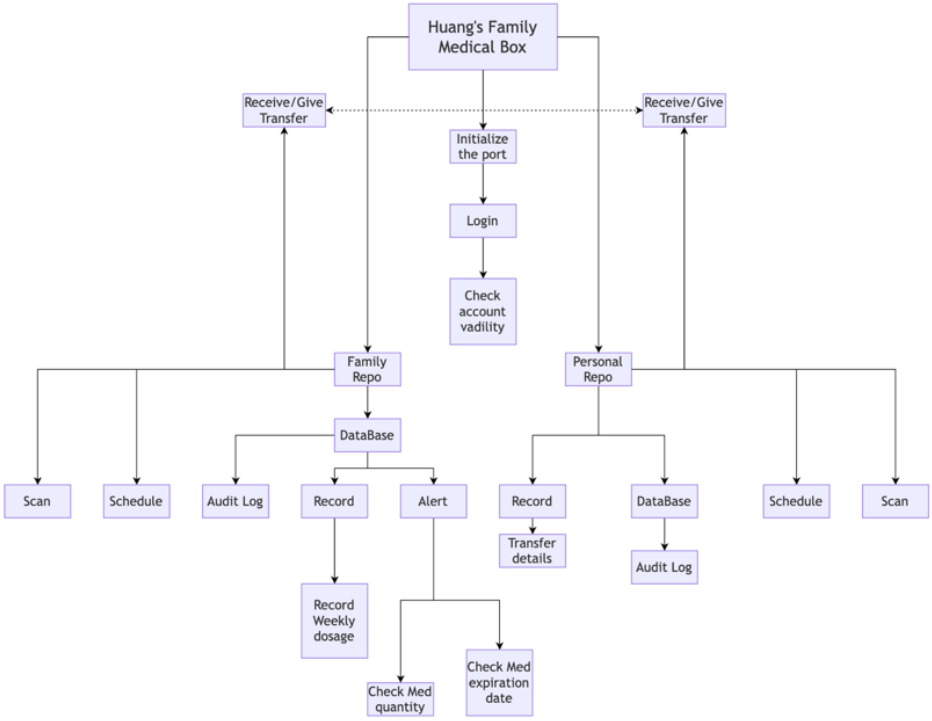


- Gantt Chart:

1	Making IA idea mindmaps																							
2	Asking for more information + gathering ideas																							
3	Present my possible solutions + specify requirements																							
4	First draft of product overview + UI diagram																							
5	presented overall design + feedback																							
6	Changing UI diagram based on feedback + flowcharts																							
7	Development timeline																							
8	Creating a wire frame diafram																							
9	GUI and repository service implementation																							
10	Asking for more requirements																							
11	Learning SQLite																							
12	UDP & TCP																							
13	Present demo. + clearify the requirements																							
14	Scanning Barcode to record medication information																							
15	Implementating the Schedule for daily medicine intake																							
16	Record for medicine intake + Audit log																							
17	Alerts system																							
18	Test																							
19	Pilot Deployment																							
20	seeking client feedback for the Pilot Deployment																							
21	Week #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

	=	Complete
	=	Late Completion

- Structure Chart:



[Word count: 148]

Criterion C: System Overview

Use Scenario:

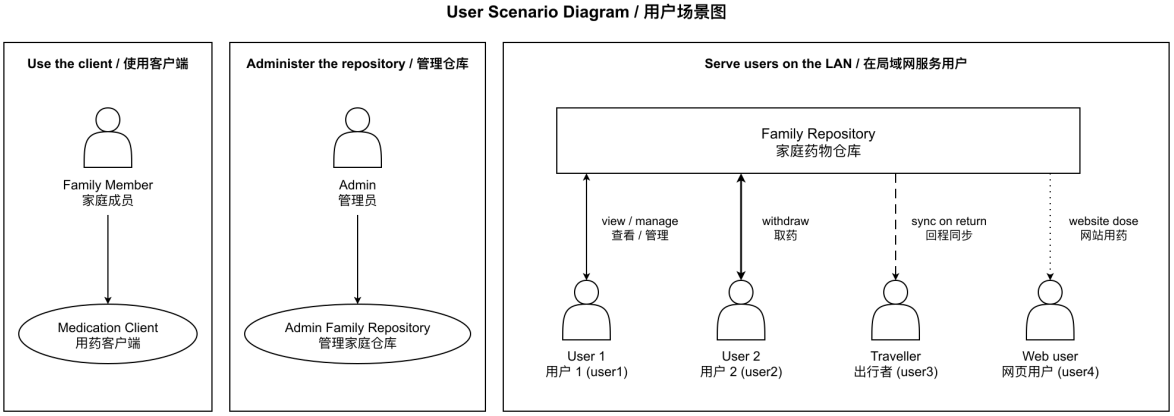


Image 1: User scenario.

Overall System Architecture Design:

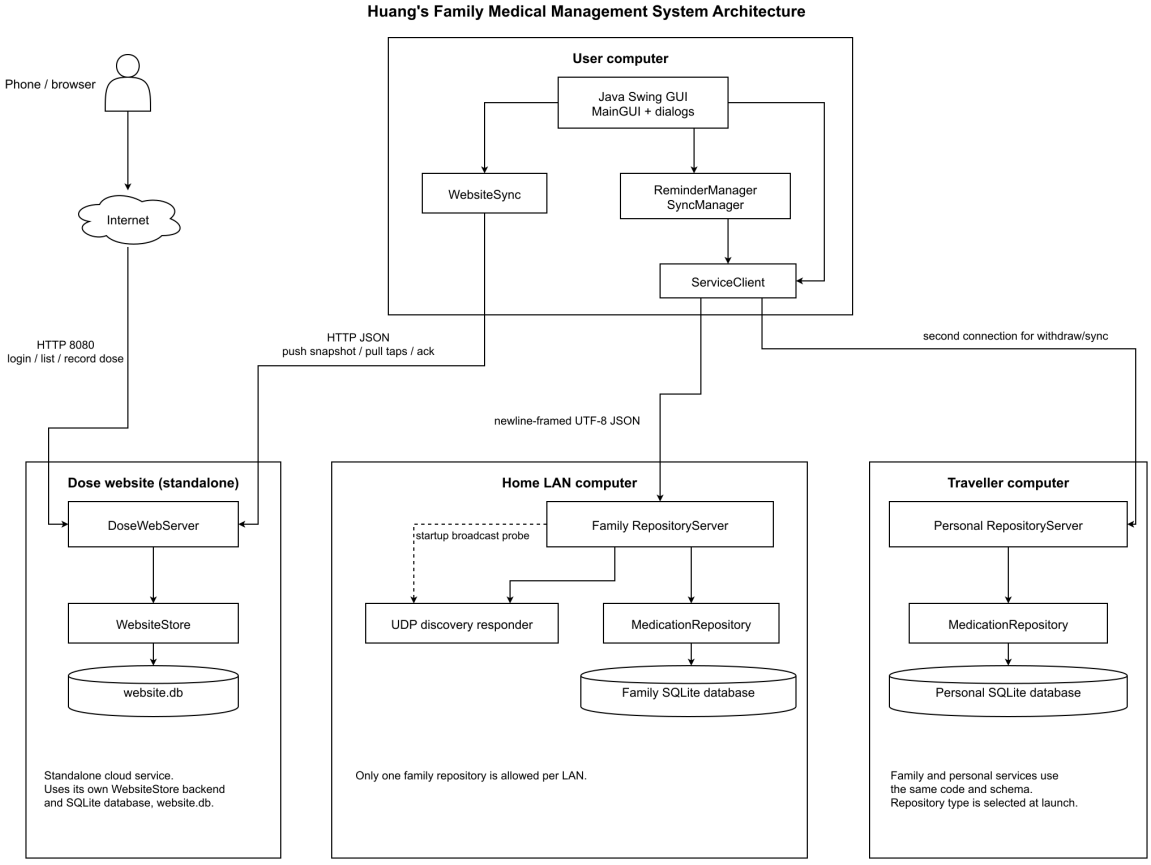


Image 2: Overall System Architecture Design

GUI:

- Design (Wireframe Sketch)



Image 3: Initial Wireframe Sketch of the GUI

DB Design:

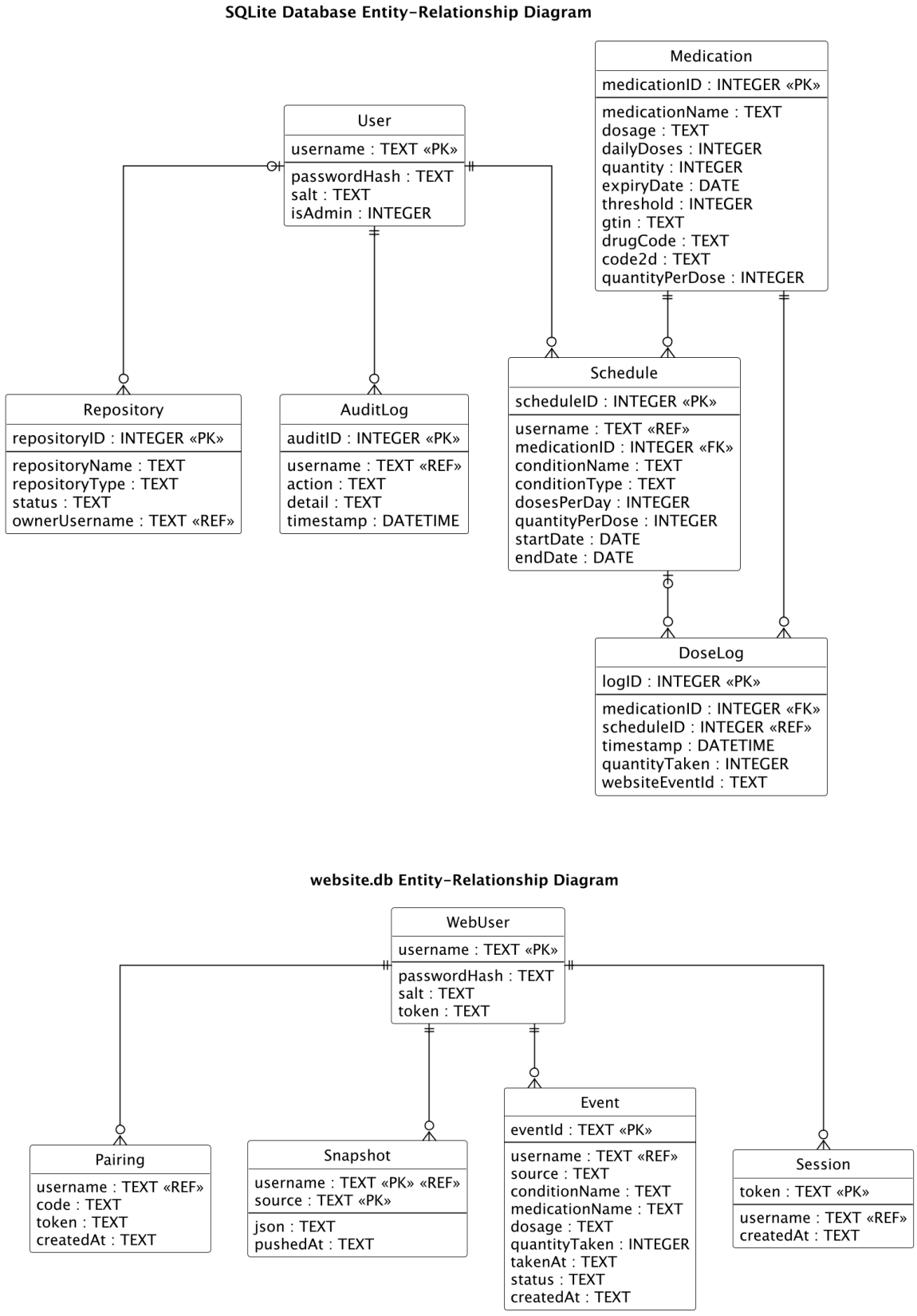


Image 4: ER diagrams for the repository DB (top) and website (bottom)

Flowcharts

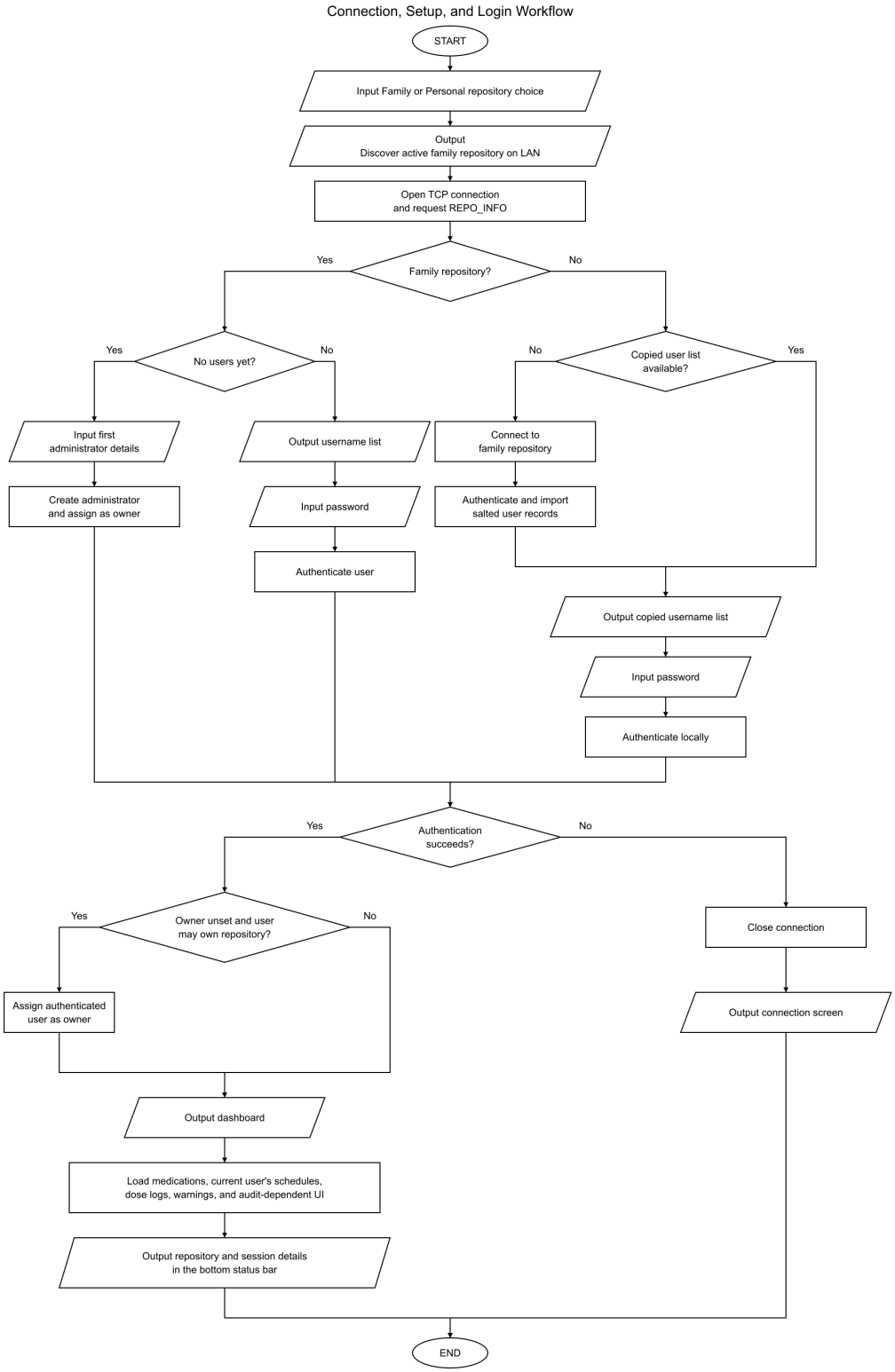


Image 5: Login workflow

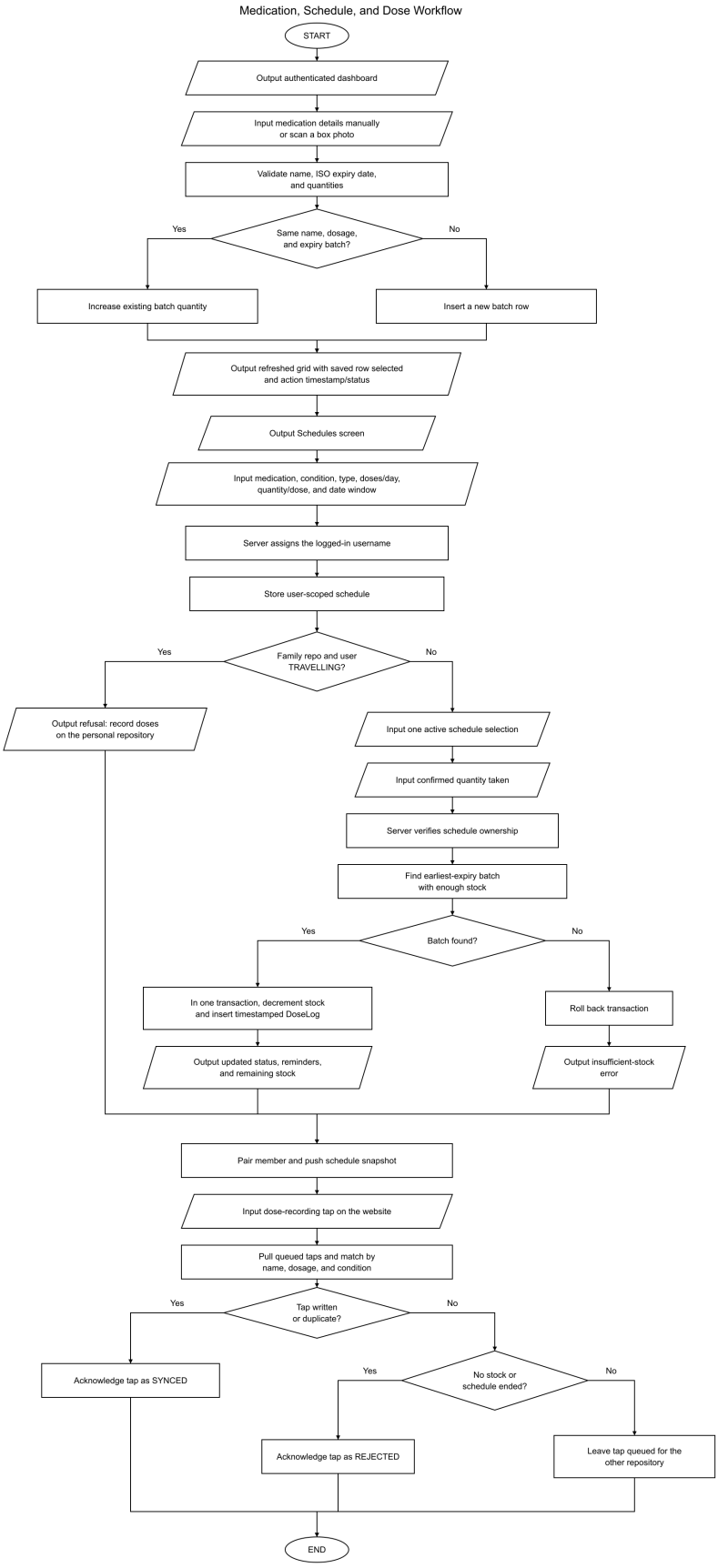


Image 6: Medication/Schedule/Dose Workflow

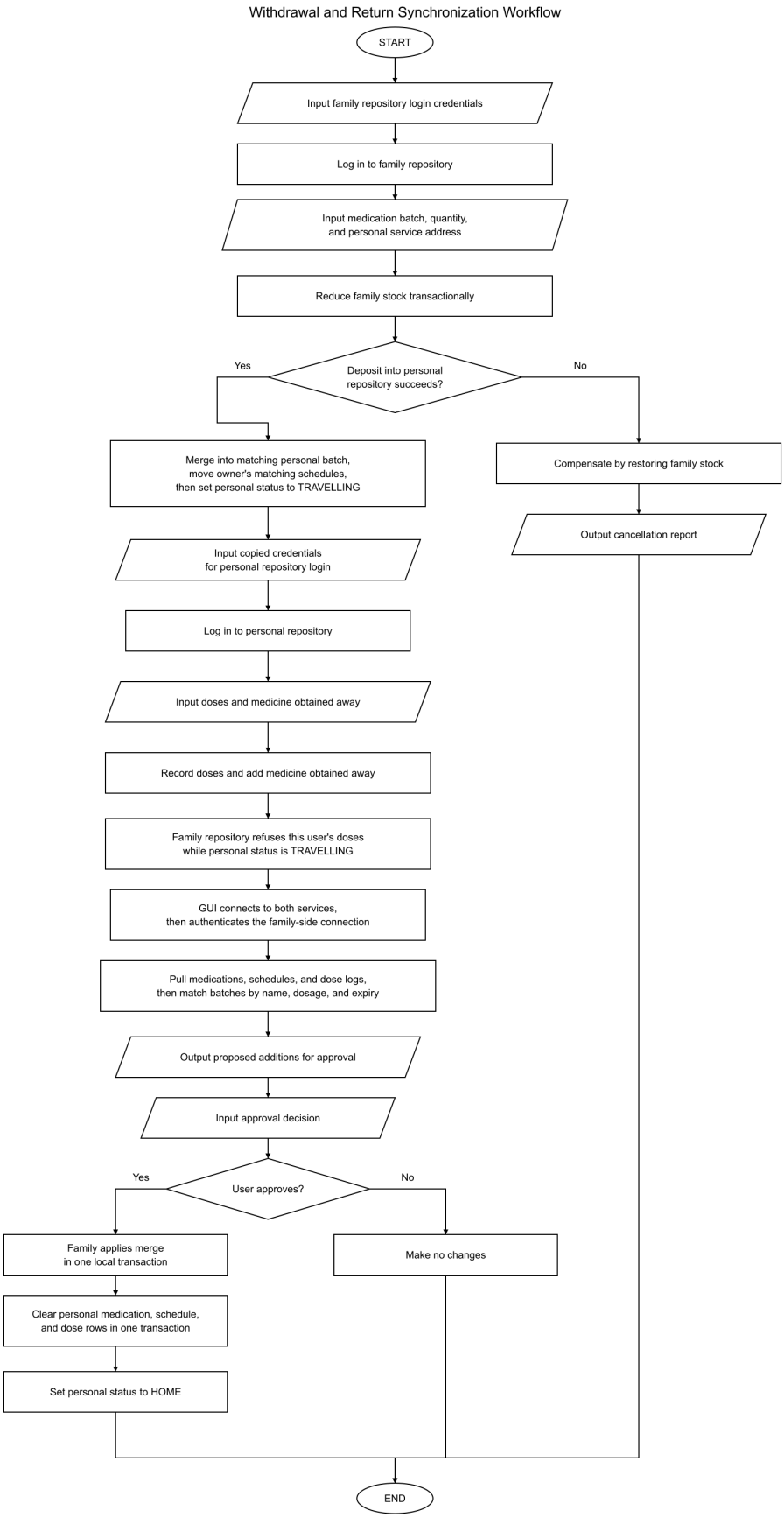


Image 7: Withdraw/Sync Workflow

Class diagrams:

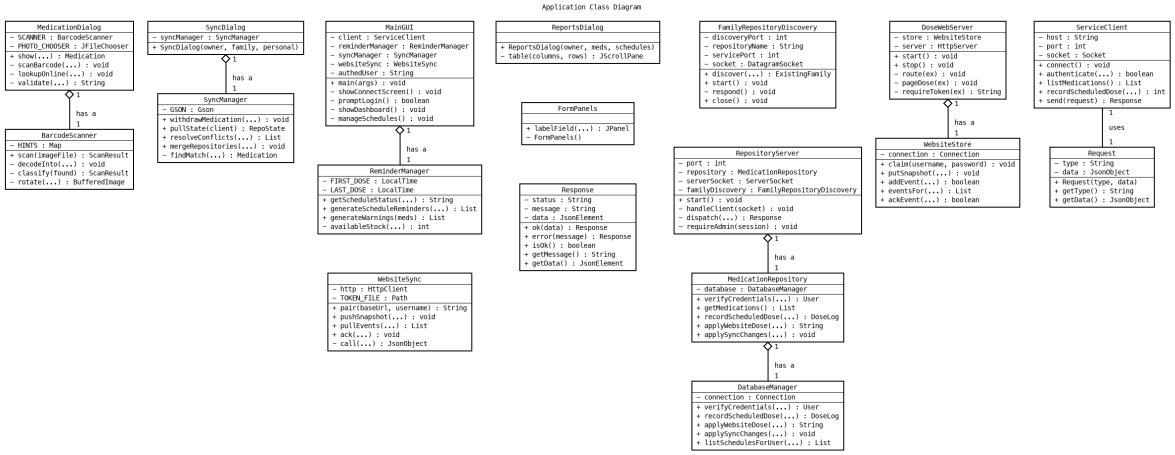


Image 8: Key application class diagrams

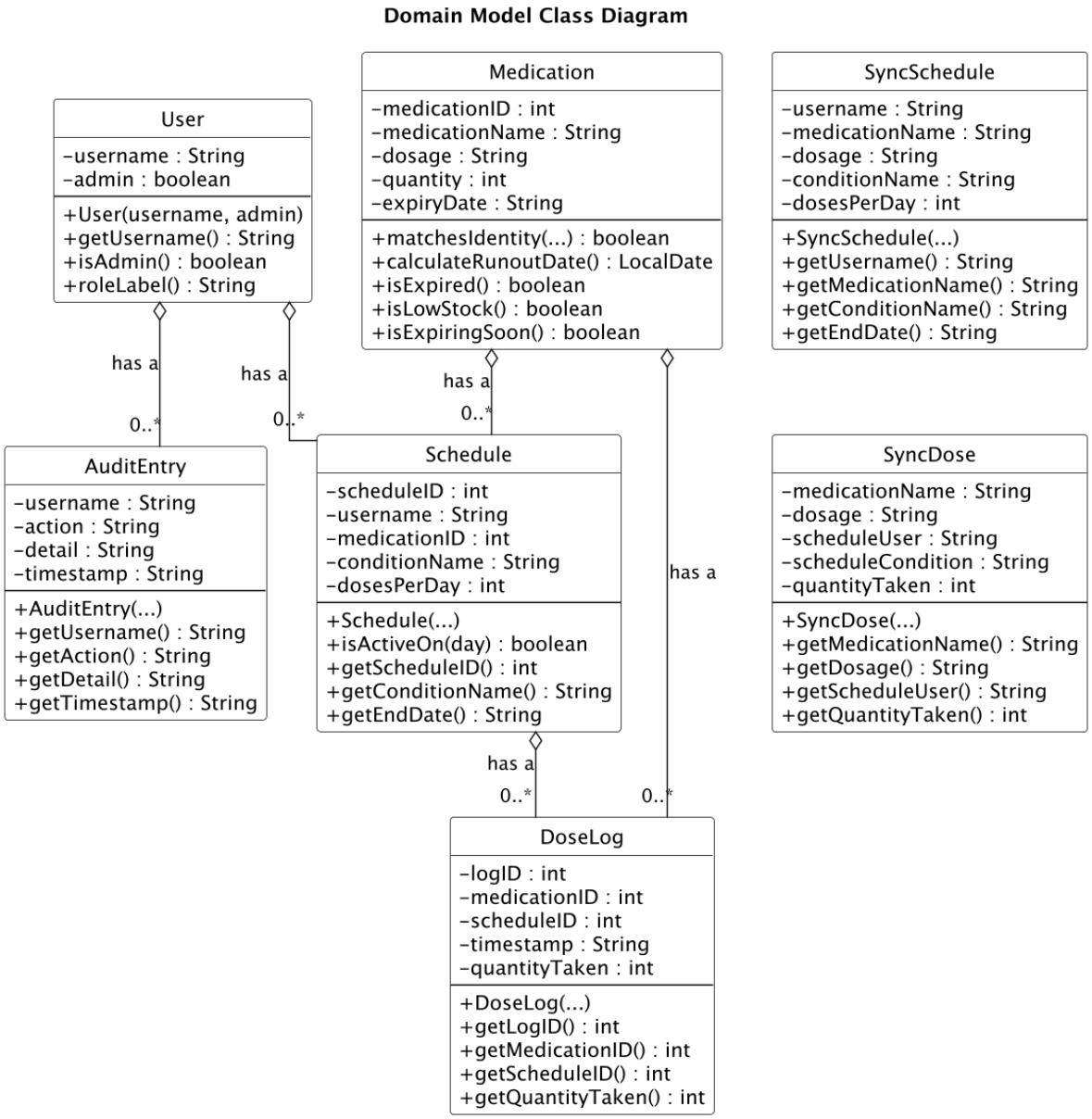


Image 9: Domain class diagrams

Algorithms as components in a system model

To stop a second family repository from starting on the same LAN:

```
Function startRepository(family)
    if family
        existing = discover(5050, 700)
        if existing is not null // another family repository service already running
            return error "a family repository is already running"
    bind TCP port //TCP service binds to TCP port 5050
    if family
        start answering probes on UDP 5050 //also listens on UDP 5050
```

For a second starting family repository or a client to discover the family repository already running and listening on UDP port 5050:

```
Function discover(port, timeoutMs) //finds existing family repository service
    targets = loopback, 255.255.255.255, and each interface broadcast
    send one probe to every target //broadcast to LAN to ask who is running
    while time remains
        packet = next UDP packet
        if packet is a probe's reply //check
            return its name, address and service port //parse the server address, port
    return null
```

To generate expiry warnings:

```
COMING_EXPIRY_DAYS = 30

Function isExpired(batch)
    if batch.expiryDate is blank
        return false
    return batch.expiryDate < today

Function isExpiringSoon(batch)
    if batch.expiryDate is blank or isExpired(batch)
        return false
    return batch.expiryDate <= today + 30 days

Function generateExpiryWarnings(medications)
    warnings = empty list
    for each batch in medications
        if isExpired(batch)
            add batch.name + " expired on " + batch.expiryDate
        else if isExpiringSoon(batch)
            add "Expires soon: " + batch.name + " on " + batch.expiryDate
    return warnings
```

To generate warnings for a schedule's medications (which can use same medication from different batches):

```
Function availableStock(schedule, medications)
    product = batch whose id is schedule.medicationID
    if product is null
        return -1
    total = 0
    for each batch in medications
        if batch.name = product.name and batch.dosage = product.dosage
            total = total + batch.quantity
    return total

Function getScheduleStatus(schedule, medications, history, now)
    if schedule is not active on today's date
        return "INACTIVE"
    taken = doses logged today for this schedule
    if taken >= schedule.dosesPerDay
        return "TAKEN"
```

```
    stock = availableStock(schedule, medications)
    if stock >= 0 and stock < schedule.quantityPerDose
        return "OUT OF STOCK"
    slotsPassed = number of today's dose times already due
    if taken < slotsPassed
        return "MISSED"
    return "DUE"
```

To withdraw medications and the user's associated schedules from family repository to a personal repository for travelling:

```
Function withdrawMedication(family, personal, medication, quantity)
    require a logged-in family member
    family.withdraw(medication.id, quantity)
    portion = copy of medication with quantity, and id 0
    try
        personalMed = personal.addMedication(portion)   // merges same name, dosage, expiry
    catch depositFailed
        family.addMedication(portion)                   // put the stock back
        return error "withdraw cancelled"
    owner = personal.ownerUsername
    for each schedule in family.listSchedules()
        if schedule.medicationID = medication.id and schedule.username = owner
            personal.addSchedule(owner, personalMed.id, schedule details)
            family.deleteSchedule(schedule.id)
    personal.status = TRAVELLING
```

To apply a web dose tap once:

```
//Repository Server: When a user sync web taps through GUI to repository server
Function applyWebsiteDose(eventId, username, condition, name, dosage, quantity, takenAt)
    begin transaction
    if a dose log already has eventId
        rollback
        return DUPLICATE
    schedule = schedule for this username, condition, name, and dosage
    if no schedule
        rollback
        return NO_SCHEDULE
    if takenAt is outside the schedule dates
        rollback
        return INACTIVE
    batch = soonest-expiring batch with enough stock
    if no batch
        rollback
        return NO_STOCK
    decrease that batch
    insert a dose log storing eventId and the web dosage-recording's takenAt
    commit
     return OK
```

To decide whether today's doses are taken timely, triggering alerts if not:

```
Function scheduleStatus(schedule, dosesToday, stock, now)
    if today is outside the start and end dates
        return INACTIVE
    taken = doses logged today for this schedule
    if taken >= dosesPerDay
        return TAKEN
    if stock < quantityPerDose
        return OUT OF STOCK
    slots = evenly spaced times from 08:00 to 22:00
    passed = number of slots at or before now
    if taken < passed
        return MISSED
     return DUE
```

To pair one member with the website:

```
//GUI side
Function pairMember(baseUrl, username)
    code = random 6-digit number
    token = POST /api/pair {username, code} // invokes registerPairing() on Web server
    save (baseUrl, username, token) on this machine
    return code

// DoseWebServer side, registering the pairing
Function registerPairing(username, code)
    if username is blank or code is blank
        return 400
    token = random 48-hex-character token
    begin transaction
        delete any pending Pairing row for this username
        insert Pairing(username, code, token, now)
        if a WebUser row already exists for this username
            replace that row's device token with this token
        end if
    commit
    return 200 {username, token, claimed}
```

To claim the pairing, the user enters the 6-digit code and create a new password:

```
//DoseWebServer side, the user enters 6-digit code and a new password to claim pairing
Function claimPairing(code, password)
    username = SELECT username FROM Pairing WHERE code = code
    if no row was found
        return "unknown or used code"
    if length of password < 4
        return "password too short"
    salt = new random salt //new salt
    token = device token saved for this username at pairing
    insert or update WebUser(username, hash(password, salt), salt, token) //save and claim!
    delete the Pairing row for this username //already claimed pairing so delete it
    sessionToken = new random token //new web browser session token
    insert Session(sessionToken, username) // new browser session
    send the browser a cookie containing sessionToken //redirect page with session cookie
```

[Word count: 142]

Test Plan

- **How to test**

SC	How to test
SC1	1. Start the family repository server. 2. Open the GUI and select Family Repository. 3. Enter localhost and port 5061, then connect. 4. Check that dad and mum appear in the username dropdown. 5. Select one user and enter an incorrect password; click Login. 6. Enter the correct password and click Login again. 7. Check whether the medication dashboard opens.
SC2	1. Start a brand-new family repository with no existing users. 2. Connect to it using the GUI. 3. Check whether the Create First Admin dialog appears. 4. Enter a username and password and create the account. 5. Open user/repository settings and check that this account is an administrator and repository owner. 6. Log out and log in again using the new account.

SC3	1. Prepare a personal repository containing imported dad and mum accounts. 2. Disconnect or stop the family repository so it cannot be contacted. 3. Open the GUI and select Personal Repository. 4. Connect to the local personal repository. 5. Confirm dad and mum appear in the username list. 6. Select mum, enter her password and log in. 7. Check that the personal repository can be used while the family service is unavailable.
SC4	1. Log in and open Add Medication. 2. Enter 阿托伐他汀钙片, dosage 20 毫克, quantity 30, expiry 2027-02-01, threshold 5 and a GTIN. 3. Save the medication. 4. Refresh/reopen the medication grid and compare every field with the entered data. 5. Add another batch with the same name, dosage and expiry. 6. Check whether it merges with the existing batch. 7. Add another batch with the same name and dosage but a different expiry. 8. Check whether a new row is created.
SC5	1. Log in as dad. 2. Create a chronic medication schedule. 3. Edit the schedule and confirm dad can save the changes. 4. Log out and log in as mum. 5. Attempt to edit Dad's schedule. 6. Attempt to delete Dad's schedule. 7. Try to create an acute schedule without entering an end date. 8. Check the system response to each invalid action.
SC6	1. Create an active schedule with 2 doses per day and sufficient stock. 2. Make sure there are no dose logs for the current day. 3. Before the first dose time, refresh the dashboard and record the displayed state. 4. After the first scheduled slot has passed, refresh again and check the state and reminder. 5. After the final scheduled slot has passed, refresh again. 6. Record the two required doses. 7. Refresh the dashboard and check the final state.
SC7	1. Add two Aspirin batches: expiry 2026-08-01, quantity 5; and expiry 2027-02-01, quantity 10. 2. Create an active schedule requiring one tablet. 3. Record the quantities before taking a dose. 4. Click Record/Take Dose once. 5. Refresh the medication grid. 6. Check the quantity of both batches. 7. Open the dose history and check that exactly one new DoseLog exists with a server timestamp.
SC8	1. Add a medication with quantity 60 and 2 daily doses. 2. Open the dashboard/report containing the run-out forecast. 3. Compare the displayed forecast with today + 30 days. 4. Add an as-needed medication with daily doses set to 0. 5. Check that no run-out forecast is shown for it. 6. Record one dose of the first medication. 7. Refresh and verify that its stock decreases only once.

SC9	1. Add at least three medication batches in an order different from their expiry-date order. 2. Make one batch already expired. 3. Set another batch's quantity to 5 and its low-stock threshold to 5. 4. Open/refresh the dashboard and reports. 5. Check whether the expired-medication warning appears. 6. Check whether the low-stock warning appears. 7. Check that the batches are displayed from earliest expiry to latest expiry.
SC10	1. Log in as mom, a normal member. 2. Attempt an administrator-only action such as changing repository settings. 3. Confirm the action is rejected. 4. Log in as dad, the administrator/owner. 5. Attempt to delete the owner account and observe the result. 6. As dad, add and then delete a test medication. 7. Inspect the stored user data and verify that no plaintext password is present and that hash/salt values are stored. 8. Open the audit log and verify username, action, detail and timestamp entries.
SC11	1. Log in to the family repository and record the current stock quantity. 2. Select a medication that has a schedule belonging to the owner of the personal repository. 3. Withdraw 8 tablets to the personal repository. 4. Open the personal repository and verify that the medication and relevant schedule were transferred. 5. Check that its status changes to TRAVELLING. 6. Reset the test data. 7. Repeat the withdrawal while deliberately causing the personal deposit to fail using the project's test/failure setup. 8. Refresh the family repository and check that the deducted stock has been completely restored.
SC12	1. Open a shared personal repository containing dad and mum and set to TRAVELLING. 2. Log in as dad and confirm successful authentication. 3. Log out and log in as mum. 4. Add a new medication named TravelMed. 5. Record a dose while using the personal repository. 6. Close the program/repository. 7. Reopen it. 8. Check that the medication, schedules and dose records are still stored locally.
SC13	1. Set the user's personal repository status to TRAVELLING. 2. Record the matching medication's current quantity in the family repository. 3. Connect to the family repository and log in as the travelling user. 4. Attempt to record a scheduled dose. 5. Observe whether the system blocks the operation. 6. Refresh the family medication data. 7. Confirm that its quantity has not changed.
SC14	1. Prepare a personal repository in TRAVELLING state with changed/new medication batches, schedules and dose logs. 2. Reconnect the computer to the home LAN. 3. Connect to the family repository. 4. Start the return/synchronization function. 5. Inspect the synchronization preview. 6. Approve the synchronization. 7. Check that matching medication batches are merged using name, dosage and expiry. 8. Check that schedules and original dose timestamps are preserved. 9. Check that the personal records are cleared and its status changes to HOME.

SC15	1. Start family repository server A on the LAN. 2. Use the GUI to confirm that server A can be discovered. 3. On the same LAN, attempt to start a second family repository server B. 4. Observe the startup result/message of server B. 5. Connect to server A again. 6. Check that server A remains available and that server B did not start.
SC16	1. Pair a user with the website through the GUI and obtain the pairing code. 2. Open the website and enter the pairing code. 3. Create/use the web password and log in. 4. Record one scheduled dose on the website and note its timestamp. 5. Return to the GUI and perform website synchronization. 6. Open the appropriate repository and verify the dose was applied to the matching schedule. 7. Check that stock decreases once and the original website timestamp is preserved. 8. Check whether the event is marked synced, rejected, or waiting. 9. Synchronize again and verify that the same event is not applied twice.
SC17	1. Start the family repository, personal repository and web server. 2. Open the GUI and test repository connection and the login dropdown. 3. Test the Create First Admin workflow on a new repository. 4. Open and use the medication grid. 5. Open Manage Users. 6. Open the schedule form, dashboard and reports. 7. Open the synchronization preview. 8. Repeat the main screens in both English and 中文 and check that the text and controls display correctly. 9. Start a synchronization, select Cancel, and verify that no repository data changes.

- **Test data and Expected results**

SC	Test/action	Test data	Expected result
SC1	Connect & log in to family repo	Host=localhost:5061 Select family repo; Choose dad/mum from dropdown; Try correct + wrong password	Dropdown shows users; Wrong password rejected; Correct password opens medication dashboard
SC2	First-run admin bootstrap	Brand-new empty family repository No users exist yet	Create first admin dialog appears New user forced to be admin + owner Normal login works afterwards
SC3	Offline personal repo login	Import dad/mum users into personal repo; Authenticate as mum while offline	Both usernames appear Mum logs in without contacting family service
SC4	Unicode batch round-trip	Name=阿托伐他汀钙片 Dosage=20 毫克 Qty=30, expiry=2027-02-01 Threshold=5 + GTIN	All fields (incl. Chinese) read back unchanged Same name+dosage+expiry merges Different expiry creates new row
SC5	User-scoped schedule CRUD	Dad creates chronic schedule Mum tries to edit/delete Dad's schedule; Acute schedule missing end date	Dad manages only his own schedules Mum is rejected Invalid acute schedule rejected

SC6	Adherence states + reminder	2 doses/day (08:00-22:00) No logs at 07:00, 12:00, 23:00 Then record two valid logs	States: DUE → MISSED → MISSED → TAKEN One missed-dose reminder issued
SC7	FEFO atomic dose recording	Aspirin batches: 2026-08-01 qty=5 2027-02-01 qty=10 Take dose of 1	Earliest batch → 4 Later batch stays 10 One server-timestamped DoseLog created
SC8	Stock + run-out forecast	Qty=60, daily doses=2 As-needed med (daily doses=0)	Forecast = today + 30 days As-needed forecast blank Dose correctly reduces stock once
SC9	Expiry & low-stock warnings	1 expired batch 1 batch qty=5 (threshold=5) 3 batches unordered by exp.	Expired + low-stock warnings shown Batches sorted earliest expiry first
SC10	Roles / hash / audit	Admin=dad, member=mum Try delete owner Change settings Add then delete test med	Unauthorized actions blocked No plaintext password (salted hash) Audit records user/action/detail/time
SC11	Withdraw + compensation	Withdraw 8 tablets + owner schedule; Simulate failed personal deposit	Success: stock+schedule move, status=TRAVELLING Failure: family quantity fully restored
SC12	Travelling personal ops	Shared personal repo (dad+mum) While TRAVELLING: add TravelMed + record dose	Both users authenticate Personal stock/schedules/doses persist locally
SC13	Travel block	Status=TRAVELLING Attempt family dose	Family dose blocked Family stock unchanged
SC14	Return sync + merge	Returning personal repo with new/changed batches, schedules, logs Approve synchronization	Preview shown; On approval: merge by name+dosage+expiry; Schedules + timestamps kept; Personal cleared → status=HOME
SC15	One family repo per LAN	Start one discoverable family server; Then try start second on same LAN	First server stays available Second refuses to start (UDP conflict)
SC16	Website dose → GUI	Paired member records dose on website GUI receives the tap	GUI applies dose once to correct repo Keeps phone timestamp Subtracts stock Marks synced / rejected / waiting
SC17	Manual GUI Test	Family and personal repoes and webserver already running, use dad and anthony	Connect, login dropdown, Create first admin, medication grid, Manage Users, schedule form, dashboard, reports, and sync preview each open in English / 中文. Cancel on the sync makes no change

Criterion D: Development

Overall structure

The product has these cooperating processes: two middler-tier repository servers (a family + a personal), a GUI client, databases and an optional HTTP dose website. A single-process desktop application would not support concurrent access by multiple family members on the home LAN, offline travel with a personal repository, or web-based dose recording.

TCP socket-communication is chosen to communicate between GUI and the repository service instead of HTTP or an RPC library because the GUI and the repository server are two Java programs on the same home network, and they only need to exchange a small set of commands. The GUI does not open the database itself, so every read and write goes through that exchange. The socket stays open, so the connection's context and does not have to be sent again with every request. Chinese names also pass through unchanged because both sides use UTF-8. HTTP would have added headers and status codes for a service that has no web pages. An RPC library such as Java RMI would hide the messages inside generated code, which would make the client/server boundary harder to explain.

Family RepositoryServer startup and client-handling mechanism

```
// product/src/net/RepositoryServer.java
94:         try {
95:             new RepositoryServer(port, new MedicationRepository(database)).start(); //new(); start()
……
106:
107:     /** Opens the listening socket and serves clients */
108:     public void start() throws IOException {
109:         boolean family = isFamilyRepository();
110:         if (family) {
111:             FamilyRepositoryDiscovery.ExistingFamily existing = null;
112:             try {
113:                 existing = FamilyRepositoryDiscovery.discover(discoveryPort,
114:                         FamilyRepositoryDiscovery.DEFAULT_TIMEOUT_MS); // active family repo on LAN?
115:             } catch (IOException e) {
116:                 System.err.println("Family repository discovery probe failed; starting anyway. / "
117:                         + "家庭仓库发现探测失败；仍将启动。 " + e.getMessage());
118:             }
119:             if (existing != null) { // already a family repository running
120:                 throw new IOException("A family repository is already running: "
121:                         + existing.name() + " at " + existing.endpoint()
122:                         + " / 家庭仓库已在运行：" + existing.name() + "，地址 "
123:                         + existing.endpoint());
124:             }
125:         }
126:
127:         serverSocket = new ServerSocket(port); //no active family repo, now open Serversocket TCP:5050
128:         try {
129:             if (family) {
130:                 try {
131:                     familyDiscovery = new FamilyRepositoryDiscovery(discoveryPort,
132:                             repository.getRepositoryName(), port); //default port 5050 for family
133:                     familyDiscovery.start(); //start listening on UDP port 5050, answering others' probes
134:                     System.out.println(logPrefix()
135:                             + "Family repository discovery active on UDP port "
136:                             + discoveryPort + " / 家庭仓库发现服务正在使用 UDP 端口 "
137:                             + discoveryPort);
……
154:             System.out.println(logPrefix() + "Repository service listening on port "
155:                     + port + " / 仓库服务正在监听端口 " + port);
156:             while (!serverSocket.isClosed()) {
157:                 Socket client = serverSocket.accept(); //each client connection gets a dedicated socket
```

```
158:                 Thread handler = new Thread(() -> handleClient(client)); //each client handled by a thread
159:                 handler.start();  // new thread start(), so not blocking the server listening thread
160:             }
```

During the service startup, to check there is no other family repository service running on the same home LAN, the starting family repository first sends one UDP probe by calling `FamilyRepositoryDiscovery.discover()`, which broadcasts to probe all LAN IP addresses' UDP port 5050.

```
// product/src/net/FamilyRepositoryDiscovery.java
59:     /** Returns the first family repository that replies, or null */
60:     public static ExistingFamily discover(int discoveryPort, int timeoutMs) throws IOException {
……
65:
66:         try (DatagramSocket probeSocket = new DatagramSocket()) { //UDP
67:             probeSocket.setBroadcast(true); //broadcast mode
68:             probeSocket.setSoTimeout(Math.max(1, timeoutMs));
69:             for (InetAddress target : discoveryTargets()) { //looping each IP on LAN
70:                 try {
71:                     probeSocket.send(new DatagramPacket(payload, payload.length,
72:                             target, discoveryPort));  //probe each LAN IP UDP port 5050
```

A reply means a family repository is already on the LAN, so an `IOException` is thrown and the startup aborts at line 120. If there is no reply, it binds the TCP port 5050 and then starts a second, daemon thread running `FamilyRepositoryDiscovery.start()` on UDP port 5050 and answers later UDP probes from other machines. This mechanism ensures only one active family repository service, which is important for data consistency and integrity.

```
// product/src/net/FamilyRepositoryDiscovery.java
97:     /** Starts UDP listening service, answering discovery probes. Only family repositories call this. */
98:     public void start() throws IOException {
99:         socket = new DatagramSocket(null); //UDP socket
100:         socket.setReuseAddress(false);
101:         socket.bind(new InetSocketAddress(discoveryPort)); //socket binds UDP port 5050
102:         responderThread = new Thread(this::respond,
103:                 "family-repository-discovery-" + discoveryPort); //thread creation
104:         responderThread.setDaemon(true); //background
105:         responderThread.start(); //UDP port 5050 listening service thread kicks off
106:     }
```

`RepositoryServer.start()` opens one `ServerSocket` (`RepositoryServer.java`, line 127) and stays in `accept()` (`RepositoryServer.java`, line 157) so it can answer next client connection. Each accepted Socket is passed to a new Thread whose job is `handleClient()` (`RepositoryServer.java`, line 158). That thread is dedicated to handle that client's requests while the listening thread immediately returns to `accept()`. So a running family RepositoryServer has the `accept()` loop over TCP port 5050 and the UDP responder (listening on UDP port 5050) at the same time.

Json format over TCP socket

The GUI and RepositoryServer communicate via Request and Response objects converted to JSON format over the wire.

```
// product/src/net/Request.java
14: public class Request {
15:
16:     private String type;
17:     private JsonObject data; //key-value pair, so use type JasonObject
18:
19:     public Request(String type, JsonObject data) {
……
29:
30:     public JsonObject getData() {
31:         JsonObject d;
32:         if (data == null)
33:             d = new JsonObject();
34:         else
35:             d = data;
36:         return d;
37:     }
38: }


// product/src/net/Response.java
17: public class Response {
……
21:     private JsonElement data; //can be object, array, one value or nothing, so use type JsonElement
22:
23:     private Response(String status, String message, JsonElement data) {
24:         this.status = status;
25:         this.message = message;
26:         this.data = data;
27:     }
28:
29:     public static Response ok(JsonElement data) {
30:         return new Response("OK", null, data);
31:     }
……
39:     public JsonElement getData() { return data; }
40: }
```

In `ServiceClient.send` (`ServiceClient.java`, lines 71–82) the client writes one JSON line and waits for one JSON line back from the server, note the conversion between request/response objects and JSON format.

```
// product/src/net/ServiceClient.java
71:    public synchronized Response send(Request request) throws IOException { //synchronized gates thread-safe
72:         out.println(GSON.toJson(request));  // convert request to Json format, send to server
73:         String line = in.readLine();   //read server response
74:         if (line == null) {
75:             throw new IOException("Connection to " + host + ":" + port + " was closed"
76:                     + " / 与 " + host + ":" + port + " 的连接已关闭");
77:         }
78:         Response response = GSON.fromJson(line, Response.class);  // convert Json to response object
79:         if (!response.isOk()) {
80:             throw new IOException(response.getMessage());
81:         }
82:         return response;
83:     }
……
130:     public List<User> listUsers() throws IOException {
131:         List<User> result = new ArrayList<>();    //list
132:         for (var el : send(new Request("LIST_USERS", null)).getData().getAsJsonArray()) {
133:             result.add(GSON.fromJson(el, User.class)); //convert Json to User object
134:         }
135:         return result;
136:     }
```

On the server side, `RepositoryServer.handleClient` (`RepositoryServer.java`, lines 214–233) reads that line, convert to a request object, dispatch to server-side processing methods, and writes the returned response object to JSON to send back to client.

```
Handle client request on a new thread
// product/src/net/RepositoryServer.java
213:     /** Serves one client connection: one JSON request per line, one response per line. */
214:     private void handleClient(Socket socket) {
215:         try (socket; //socket IO
216:              BufferedReader in = new BufferedReader(  //reader
217:                      new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
218:              PrintWriter out = new PrintWriter(    // writer
219:                      socket.getOutputStream(), true, StandardCharsets.UTF_8)) {
220:             Session session = new Session(); // auth state for this connection
221:             String line;
```

```
222:             while ((line = in.readLine()) != null) { //line by line read
223:                 Response response;
224:                 try {
225:                     Request request = GSON.fromJson(line, Request.class); //Json convert to request object
226:                     response = dispatch(request, session);  //distribute to different processing methods
227:                 } catch (Exception e) {
228:                     // a bad request should never bring the service down
229:                     String detail = e.getMessage() == null
230:                             ? e.getClass().getSimpleName() : e.getMessage();
231:                     response = Response.error("Request failed / 请求失败: " + detail);
232:                 }
233:                 out.println(GSON.toJson(response)); //response convert to Json to send back to client
234:             }
235:         } catch (IOException e) {
236:             // client disconnected mid-request; nothing to clean up; every
237:             // resource is try-with-resources
238:         }
239:     }
```

A binary protocol or Java object serialisation would have been slightly faster, but JSON is chosen because it is human-readable, language-neutral and easy to debug.

JDBC, Statement, PreparedStatement, Transaction, Synchronized method

The following code has `Statement`, `PreparedStatement` and `Transaction` lines in it. Fixed schema statements use `Statement` objects (lines 277-278). `PreparedStatement` supports placeholders to be replaced with variables (lines 280-287). `Transaction` ensures atomicity of the scoped operations, either they all succeed or all fail, no partial success: starts with `connection.setAutoCommit(false)` at line 275, then `commit()` at line 290, `rollback()` at line 292, and `connection.setAutoCommit(true)` at line 295. The keyword `synchronized` dictates only one thread can run the method at one time.

```
// product/src/db/DatabaseManager.java
274:     public synchronized void importUsers(List<String[]> users) throws SQLException {
275:         connection.setAutoCommit(false);  //transaction setup
276:         try {
277:             try (Statement st = connection.createStatement()) {
278:                 st.execute("DELETE FROM User");        //Statement
279:             }
280:             try (PreparedStatement ps = connection.prepareStatement("INSERT INTO User "
281:                     + "(username, passwordHash, salt, isAdmin) VALUES (?, ?, ?, ?)")) {
282:                 for (String[] u : users) {
283:                     ps.setString(1, u[0]);
284:                     ps.setString(2, u[1]);
285:                     ps.setString(3, u[2]);
286:                     ps.setInt(4, "1".equals(u[3]) ? 1 : 0);
287:                     ps.executeUpdate();
288:                 }                                  //PreparedStatement
289:             }
290:             connection.commit();              //Transaction Commit
291:         } catch (SQLException e) {
292:             connection.rollback();       // rollback
293:             throw e;
294:         } finally {
295:             connection.setAutoCommit(true);    //reset
296:         }
297:     }
```

List<>, ArrayList<>(), java.util.Map<,>, java.util.HashMap<>()

```
// product/src/net/RepositoryServer.java
518:                 java.util.Map<Integer, Schedule> schedulesById = new java.util.HashMap<>();
519:                 for (Schedule s : repository.getSchedules()) {
520:                     schedulesById.put(s.getScheduleID(), s);
521:                 }
522:                 List<SyncDose> doses = new ArrayList<>();
523:                 for (DoseLog log : repository.getDoseHistory(0)) {
524:                     Medication m = repository.getMedication(log.getMedicationID());
525:                     if (m == null) {
526:                         continue;
527:                     }
528:                     Schedule s = schedulesById.get(log.getScheduleID());
529:                     doses.add(new SyncDose(m.getMedicationName(), m.getDosage(),
```

`HashMap` enables fast lookup of value given a key, while an `ArrayList` enables accessing its elements by index. `HashMap` does not allow duplicate keys, but `ArrayList` does. The former does not maintain the insertion order of elements but the later does. Here both are used, for schedules should not have duplicated IDs, but syncDoses can have identical ones.

Salted SHA-256 hashing of passwords

```
// product/src/auth/PasswordHasher.java
20: public class PasswordHasher {
21:
22:     private static final SecureRandom RANDOM = new SecureRandom();
23:     private static final HexFormat HEX = HexFormat.of();
24:
25:     /** random salt, as a hex string. */
26:     public static String newSalt() {
27:         byte[] salt = new byte[16];
28:         RANDOM.nextBytes(salt); //randomizd salt
29:         return HEX.formatHex(salt);
30:     }
31:
32:     /** hashing. */
33:     public static String hash(String password, String saltHex) {
34:         try {
35:             MessageDigest digest = MessageDigest.getInstance("SHA-256");
36:             digest.update(HEX.parseHex(saltHex)); // add salt
37:             return HEX.formatHex(digest.digest(password.getBytes(StandardCharsets.UTF_8))); //hashing
38:         } catch (NoSuchAlgorithmException e) {
39:             throw new IllegalStateException("SHA-256 is required but unavailable", e);
40:         }
41:     }
42:
43:     /** True only if password hashed with the salt ==  stored hash. */
44:     public static boolean verify(String password, String saltHex, String hashHex) {
45:         return hash(password, saltHex).equals(hashHex); //compared hashed values
46:     }
47: }
```

User passwords are encrypted using SHA-256 hashing function with a 16-byte salt and store on the server-side. A user-entered password must be subjected to the same hashing with the same salt and match the stored hashed value to be considered valid. No plain-text password stored.

SwingWorker, networking work off the GUI thread

```
// product/src/gui/MainGUI.java
330:         new SwingWorker<ExistingFamily, Void>() { // instantiate new instance
331:             @Override
332:             protected ExistingFamily doInBackground() throws Exception { // implementation bg task logic
333:                 ExistingFamily fam = FamilyRepositoryDiscovery.discover(
334:                         FamilyRepositoryDiscovery.DEFAULT_DISCOVERY_PORT,
335:                         FamilyRepositoryDiscovery.DEFAULT_TIMEOUT_MS);
336:                 if (fam != null) {
337:                     fam.address().getHostName(); // warm the reverse-DNS cache off the EDT
338:                 }
339:                 return fam; //return value
340:             }
341:
342:             @Override
343:             protected void done() { // after the task is done, then do the following
344:                 searchAgain.setEnabled(true);
345:                 ExistingFamily fam;
346:                 try {
347:                     fam = get();  //returned value of doInBackground()
……
366:         }.execute(); // kick off the swingWorker thread
```

Line 330 instantiates the `SwingWorker` instance, whose `execute()` hands this instance to Swing's shared worker thread pool. The networking task in `doInBackground()` runs on one

of `SwingWorker`'s threadpool threads, and `done()` is then run on the `Swing event` thread so it can update the GUI label and fields. All of these works are done without blocking the Main GUI thread, so the UI remains responsive to user interactions.

Lambda Exmpression

```
// product/src/gui/MainGUI.java
210:         personalChoice.addActionListener(e -> {
211:             host.setText("localhost");
212:             port.setValue(net.RepositoryServer.DEFAULT_PERSONAL_PORT);
213:         });
214:         familyChoice.addActionListener(e -> {
215:             port.setValue(net.RepositoryServer.DEFAULT_FAMILY_PORT);
216:             fillFromDiscovered(found.get(), host, port);
217:         });
218:         searchAgain.addActionListener(e ->
219:                 discoverFamilyRepository(familyStatus, host, port, familyChoice, found, searchAgain));
```

These three lambdas are click handlers on the connect screen. They are a strong choice because each handler stays right next to the control it belongs to, keeping the UI wiring short, readable, and easy. They also capture local variables like host, port, found, and searchAgain directly, which a separate method or class could not do without extra parameters or fields.

Building HTML content in java

```
// product/web/src/DosePages.java
125:     private static String section(String title, String source, String json) {
126:         StringBuilder b = new StringBuilder("<h2>").append(title).append("</h2>");
127:         if (json == null || json.isBlank() || "[]".equals(json.trim())) {
128:             b.append("<div class='sub'>(not pushed yet / 尚未推送)</div>");
129:             return b.toString();
130:         }
……
148:             b.append("""
149:                     <tr><td>%s %s</td><td>%s</td><td>%s</td><td>%s</td>
150:                     <td><form method="post" action="/dose">
151:                       <input type="hidden" name="source" value="%s">
152:                       <input type="hidden" name="conditionName" value="%s">
153:                       <input type="hidden" name="medicationName" value="%s">
154:                       <input type="hidden" name="dosage" value="%s">
155:                       <input type="hidden" name="quantityTaken" value="%s">
156:                       <button type="submit">Record / 记录</button>
157:                     </form></td></tr>
……
162:         b.append("</table>");
163:         return b.toString();
```

To support HTML page rendering and subsequent user post action, a `StringBuilder` object b is used to append all the html elements including the form input elements. `StringBuilder` is chosen because it is mutable and efficiently support dynamic growth of the intended string length.

Exception handling

```
// product/src/net/RepositoryServer.java
127:         serverSocket = new ServerSocket(port);
128:         try {
129:             if (family) {
130:                 try {
131:                     familyDiscovery = new FamilyRepositoryDiscovery(discoveryPort,
132:                             repository.getRepositoryName(), port);
133:                     familyDiscovery.start();
```

```
134:                     System.out.println(logPrefix()
135:                             + "Family repository discovery active on UDP port "
136:                             + discoveryPort + " / 家庭仓库发现服务正在使用 UDP 端口 "
137:                             + discoveryPort);
138:                 } catch (SQLException e) {
139:                     throw new IOException("Could not read the family repository name. / "
140:                             + "无法读取家庭仓库名称。", e);
141:                 } catch (BindException e) {
142:                     throw new IOException("Could not reserve the family discovery port "
143:                             + discoveryPort + "; another family repository may be starting. / "
144:                             + "无法占用家庭仓库发现端口 " + discoveryPort
145:                             + "；可能有另一个家庭仓库正在启动。", e);
146:                 } catch (IOException e) {
147:                     familyDiscovery.close();
148:                     familyDiscovery = null;
149:                     System.err.println("Family repository discovery responder failed; "
150:                             + "starting without LAN enforcement. / 家庭仓库发现响应服务失败；"
151:                             + "将在不进行局域网唯一性检查的情况下启动。 " + e.getMessage());
152:                 }
153:             }
……
160:             }
161:         } finally {
162:             if (familyDiscovery != null) {
163:                 familyDiscovery.close();
164:             }
165:             if (serverSocket != null && !serverSocket.isClosed()) {
166:                 serverSocket.close();
167:             }
168:         }
```

Here the `SQLException` and a `BindException` during family discovery are wrapped and escalated to `IOException` with clear bilingual messages, while a generic `IOException` is treated as non-fatal, the `familyDiscovery` responder is closed, a warning is logged, and the server continues. The outer `try/finally` ensures the responder and `serverSocket` are closed on any exit path and preventing resource leaks.

Automated Testing as a complement to Manual Testing

The main application is highly user-interactive, so main testing effort will be doing UI-driven manual tests on all the buttons, menus, controls, according to the success criteria and test plan in criterion A and C. In addition, I have developed a test program to test the repository service.

```
// product/src/SmokeTest.java
120:     private static void run(ServiceClient family, ServiceClient personal,
121:                             ServiceClient personalB) throws Exception {
122:         ReminderManager reminders = new ReminderManager();
123:         SyncManager sync = new SyncManager();
124:         String today = LocalDate.now().toString();
125:
126:         // --- i, ii: create medication profiles with quantity, dosage, expiry ----
127:         Medication aspirin = family.addMedication(new Medication(
128:                 0, "Aspirin", "100mg", 2, 20, "2027-02-01", 5), false);
129:         family.addMedication(new Medication(
130:                 0, "Lipitor", "20mg", 1, 5, "2025-12-01", 5), false);
131:         family.addMedication(new Medication(
132:                 0, "Metoprolol", "50mg", 2, 60, "2026-08-15", 10), false);
133:         check("SC4: medication profile created with a generated ID",
134:                 aspirin.getMedicationID() > 0);
135:         Medication stored = find(family.listMedications(), "Aspirin");
136:         check("SC4: quantity, dosage and expiry stored",
137:                 stored.getQuantity() == 20 && stored.getDosage().equals("100mg")
138:                         && stored.getExpiryDate().equals("2027-02-01"));
….
```

[Word count: 964]

Bibliography / External Libraries

(*The following third-party libraries are used by the production and distributed in the product's lib/ Standard Java libraries are not listed.)

Google. (2024). Gson (Version 2.11.0) [Computer software]. https://github.com/google/gson https://repo1.maven.org/maven2/com/google/code/gson/gson/2.11.0/gson-2.11.0.jar

Google. (2024). ZXing Core (Version 3.5.4) [Computer software]. https://github.com/zxing/zxing https://repo1.maven.org/maven2/com/google/zxing/core/3.5.4/core-3.5.4.jar

Google. (2024). ZXing JavaSE (Version 3.5.4) [Computer software]. https://github.com/zxing/zxing https://repo1.maven.org/maven2/com/google/zxing/javase/3.5.4/javase-3.5.4.jar

QOS.ch. (2024). SLF4J API (Version 2.0.13) [Computer software]. https://www.slf4j.org/ https://repo1.maven.org/maven2/org/slf4j/slf4j-api/2.0.13/slf4j-api-2.0.13.jar

QOS.ch. (2024). SLF4J NOP (Version 2.0.13) [Computer software]. https://www.slf4j.org/ https://repo1.maven.org/maven2/org/slf4j/slf4j-nop/2.0.13/slf4j-nop-2.0.13.jar

Xerial. (2025). sqlite-jdbc (Version 3.49.1.0) [Computer software]. https://github.com/xerial/sqlite-jdbc https://repo1.maven.org/maven2/org/xerial/sqlite-jdbc/3.49.1.0/sqlite-jdbc-3.49.1.0.jar

Criterion E: Evaluation

Meeting the success criteria

The product was evaluated against the success criteria in criterion A using the Criterion C test plan, manual testing, as well as the automated SmokeTest program in criterion D.

SC	Test/action	Test data	Expected result	Test Pass
SC1	Connect & log in to family repo	Host=localhost:5061 Select family repo; Choose dad/mum from dropdown; Try correct + wrong password	Dropdown shows users; Wrong password rejected; Correct password opens medication dashboard	Pass
SC2	First-run admin bootstrap	Brand-new empty family repository No users exist yet	Create first admin dialog appears New user forced to be admin + owner Normal login works afterwards	Pass
SC3	Offline personal repo login	Import dad/mum users into personal repo; Authenticate as mum while offline	Both usernames appear Mum logs in without contacting family service	Pass
SC4	Unicode batch round-trip	Name=阿托伐他汀钙片 Dosage=20 毫克 Qty=30, expiry=2027-02-01 Threshold=5 + GTIN	All fields (incl. Chinese) read back unchanged Same name+dosage+expiry merges Different expiry creates new row	Pass
SC5	User-scoped schedule CRUD	Dad creates chronic schedule Mum tries to edit/delete Dad's schedule; Acute schedule missing end date	Dad manages only his own schedules Mum is rejected Invalid acute schedule rejected	Pass
SC6	Adherence states + reminder	2 doses/day (08:00-22:00) No logs at 07:00, 12:00, 23:00 Then record two valid logs	States: DUE → MISSED → MISSED → TAKEN One missed-dose reminder issued	Pass
SC7	FEFO atomic dose recording	Aspirin batches: 2026-08-01 qty=5 2027-02-01 qty=10 Take dose of 1	Earliest batch → 4 Later batch stays 10 One server-timestamped DoseLog created	Pass
SC8	Stock + run-out forecast	Qty=60, daily doses=2 As-needed med (daily doses=0)	Forecast = today + 30 days As-needed forecast blank Dose correctly reduces stock once	Pass
SC9	Expiry & low-stock warnings	1 expired batch 1 batch qty=5 (threshold=5) 3 batches unordered by exp.	Expired + low-stock warnings shown Batches sorted earliest expiry first	Pass
SC10	Roles / hash / audit	Admin=dad, member=mum Try delete owner Change settings Add then delete test med	Unauthorized actions blocked No plaintext password (salted hash) Audit records user/action/detail/time	Mostly Pass, non-admin action needs more restriction

SC11	Withdraw + compensation	Withdraw 8 tablets + owner schedule; Simulate failed personal deposit	Success: stock+schedule move, status=TRAVELLING Failure: family quantity fully restored	Pass
SC12	Travelling personal ops	Shared personal repo (dad+mum) While TRAVELLING: add TravelMed + record dose	Both users authenticate Personal stock/schedules/doses persist locally	Pass
SC13	Travel block	Status=TRAVELLING Attempt family dose	Family dose blocked Family stock unchanged	Pass
SC14	Return sync + merge	Returning personal repo with new/changed batches, schedules, logs Approve synchronization	Preview shown; On approval: merge by name+dosage+expiry; Schedules + timestamps kept; Personal cleared → status=HOME	Pass
SC15	One family repo per LAN	Start one discoverable family server; Then try start second on same LAN	First server stays available Second refuses to start (UDP conflict)	Pass
SC16	Website dose → GUI	Paired member records dose on website GUI receives the tap	GUI applies dose once to correct repo Keeps phone timestamp Subtracts stock Marks synced / rejected / waiting	Pass
SC17	Manual GUI Test	Family and personal repoes and webserver already running, use dad and anthony	Connect, login dropdown, Create first admin, medication grid, Manage Users, schedule form, dashboard, reports, and sync preview each open in English / 中文. Cancel on the sync makes no change	Mostly-Pass, help-text is not user-friendly, user dosage-uptake history feature incomplete

In addition, 83 automated test cases all passed:



Image 1. SmokeTest output

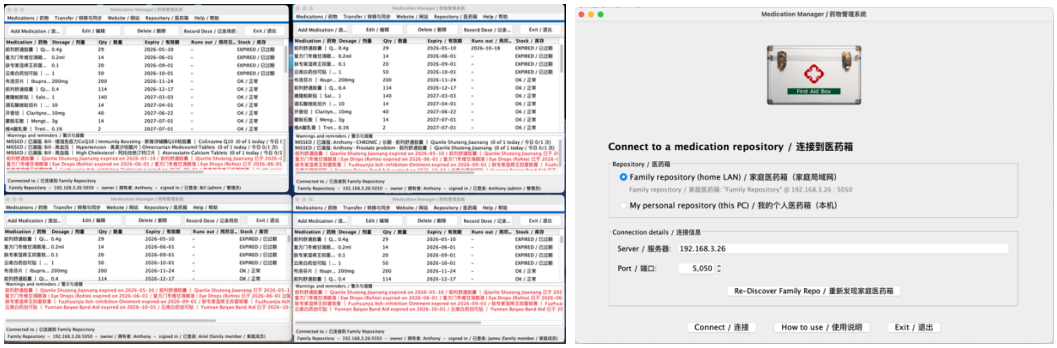


Image 2. Four clients on a different machine
connecting to the family server via auto-discovery

Furthermore, to do load testing, I opened 4 GUI clients from a different machine connecting to the family repository server, tried various meds, schedules, withdraw, sync, reports, audit, etc, within a time of 20 minutes on 4 clients. The server's memory and CPU showed virtually no pressure. So our repository servers use light resource in terms of memory and CPU. The disk usage is minimal as well since the family meds data is not much (family.db size ~=44 KB).

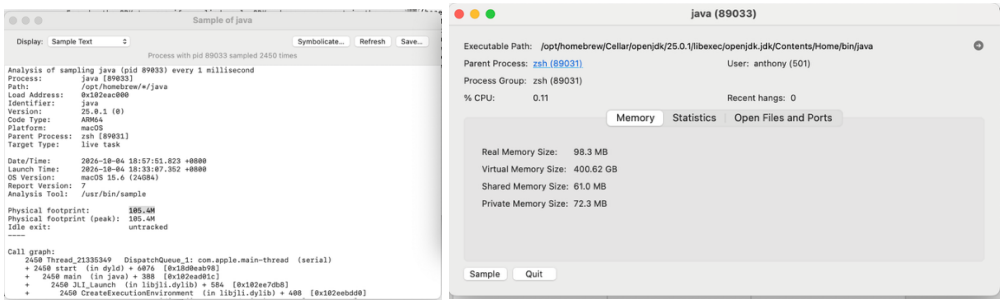


Image 3. Four clients' busy UI actions led to
a memory footprint of 105.4 M and no CPU pressure

Client feedback

Deployment in the clients' home LAN has been successful and pilot use so far has been smooth. Mr. and Mrs. Huang confirmed that the daily workflow (login, record dose, see warnings, withdraw for travel, sync on return) matches the problems they originally described. They valued the bilingual interface and the fact that core medication data never leaves the home LAN. They noted that the website pairing step felt slightly cumbersome the first time, but accepted the security rationale once explained. They like the option of web-entry of dosages.

Recommendations for further development

While overall the product satisfies the success criteria agreed with the client and is ready for

daily use on the home LAN. There are some rooms for improvement:

- Restriction to non-admin users on certain operations like changing repository settings, etc. Help content can be more user-friendly. Dosage-uptake history can enhance the user experience.
- Currently the server and client are launched by terminal commands/scripts, would be more user friendly to have them just click-and-run.
- The repository service can be changed to be a background system service instead of a user-launched service (when user logs out, it dies).
- Currently phone tapping of dosage is through a web URL in phone's browser, a smart phone app will be a cool addition.
- The repository and GUI have not been extensively tested on windows, due to machine resource scarcity, since java is cross-platform, they should work. But would be good to verify.
- The DoseWebServer currently has been tested as a http server on linux and Mac at home, but have not been deployed to real cloud server, due to budget and various difficulty of getting a https certificate and a real domain name for server with a real IP. Would be good to try that one day.

[Word Count: 398]